Adaptive Air Hockey AI

物理予測と強化学習を組み合わせ、相手と環境に適応するエアホッケーロボット

Physical AI 応用1 最終課題 ／ OmniCampus アカウント: yoichiozaki ／ 2026 年 10 月

コード: https://github.com/yoichiozaki/physical-ai-robot-sports （air-hockey/） ／ 動画: YouTube「PAI最終課題_yoichiozaki」

1. 背景と目的

エアホッケーは、2–5 m/s で壁に反射しながら動くパックの軌道を 0.3–0.6 秒で予測し、速度・加速度・可動範囲に限界のある機械で打ち返す必要がある。物理モデルによる予測制御は精密だが「どこへどう打つか」という戦略は人手設計に頼り、相手や環境が変わると最適でなくなる。一方、強化学習は戦略を経験から獲得できるが、物理を一から学ぶと試行回数が膨大になる。本研究では、MuJoCo 上に関節とアクチュエータで動く 2 軸ガントリーロボットを構築し、物理予測制御を土台に、強化学習による戦略選択とオンライン同定による予測補正を組み合わせた「適応型」エアホッケー AI を開発し、従来方式と定量比較した。

研究課題: 従来の物理モデルによる予測制御と比較して、機械学習を用いた戦略選択・予測補正は、対戦性能と未知環境への適応能力を改善できるか？

2. タスク概要

MuJoCo 上の 2.0 m × 1.0 m のテーブル（ゴール幅 0.30 m）で、自ロボット（青）と相手ロボット（赤）が 60 秒の試合を行う。ゴール後は失点側からフェイスオフ、パックが片側に 7 秒以上留まった場合は相手側から再開する（ポゼッション規則）。相手は 5 種（静止・ランダム・追従型・守備型・攻撃型）で、人間らしさのため反応遅延（0.08–0.18 s）と誤った物理モデルを持つ。

3. システム構成

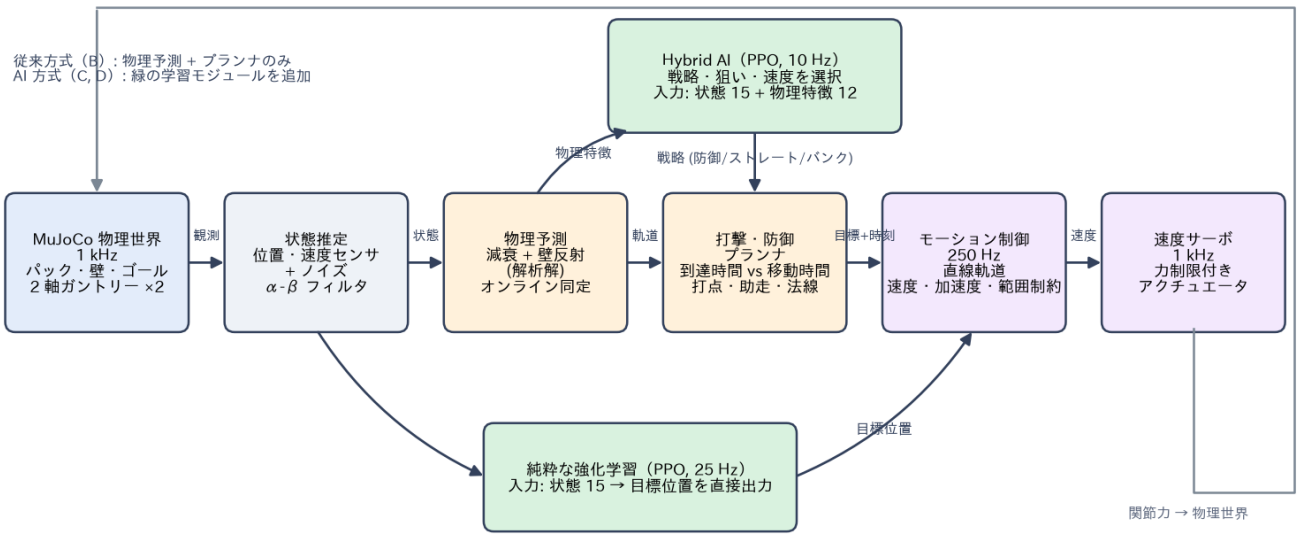


図1 システム構成。判断 50 Hz・軌道生成 250 Hz・速度サーボ 1 kHz の多重レート構成。緑が学習モジュール。

シミュレーション・観測・制御器・学習・評価・可視化・クラウド実行をパッケージ単位で疎結合にし、すべての制御器（従来方式・純粋 RL・Hybrid）の出力を「マレットの目標位置」に統一した。行動は必ずモーション制御（速度・加速度・可動範囲制約）を経由して関節アクチュエータに渡るため、学習アルゴリズムを替えても物理シミュレータの変更は不要である。

4. 物理シミュレーション

パックはエアクッションで浮上するため、x・y のスライド関節で平面拘束し、粘性減衰（減衰率 c = 0.15 /s）で空気抵抗と残留摩擦を表した。衝突形状は同じ高さの球にして接触法線を厳密に水平にし、接触は明示的な接触ペア（法線方向のみ）に限定した。MuJoCo の dampratio 方式では剛性が $1/(\tau\zeta)^2$ に比例し、反発係数 0.85（$\zeta \approx 0.05$）が 500 Hz で数値的に発散したため、固有振動数 $\omega = 600$ rad/s を固定して $\tau = 1/(\omega\zeta)$ とする方式を考案し、1 kHz で目標反発係数を再現した（表1）。

目標反発係数	実測 平均	標準偏差	範囲	試行
0.60	0.595	0.021	0.581–0.642	15
0.75	0.746	0.001	0.744–0.749	15
0.85	0.847	0.005	0.841–0.856	15
0.95	0.941	0.008	0.930–0.956	15

表1 壁反発係数の校正結果（速度 0.5–6 m/s、入射角 30–90°）。

ロボットは実機のエアホッケーロボットで主流の 2 軸直交ガントリー（X: ブリッジ 1.6 kg、Y: キャリッジ 0.4 kg）で、各軸はスライド関節と力制限付き速度サーボ（1 kHz）で駆動する。速度上限 3 m/s、加速度上限 X 15 / Y 25 m/s^2、可動範囲は自陣のみ（中央線を越えない）。物理テスト 8 種（静止安定・壁反射・マレット衝突・8 m/s での貫通なし・可動範囲・数値安定・ゴール判定・固定シード再現）を自動化した。

5. ロボット制御（物理予測制御 = 従来方式 B）

パック軌道は区間ごとに解析解 $v(t) = v_0 e^{-ct}$, $p(t) = p_0 + v_0(1-e^{-ct})/c$ で求め、壁到達時刻を閉形式で計算して法線成分を $-e$ 倍する（予測誤差: 0.25 s 先 1.8 cm、0.5 s 先 2.1 cm）。打撃では、パックを方向 d・速さ μ で打ち返すための接触法線 $n = (\mu d - v_{in})/|\mu d - v_{in}|$ と、法線方向のマレット速度 $s_n = v_{in}\cdot n + |\mu d - v_{in}|/(1+e_m)$ を求める。予測軌道上の各候補点について「パック到達時刻」と「ロボットの停止到達時間 + 助走時間」を比較し、到達不可能な打点は選ばない。バンクショットは反発で傾きが $-e$ 倍になることを用いた閉形式で狙う。

実装上の要点は、(1) 判断周期 50 Hz では打撃開始が最大 20 ms ずれ打点誤差 4 cm（方向誤差 10–30°）になったため、低レベル側で指定時刻に目標を切り替える「時刻付き打撃指令」を導入、(2) 軸ごとに加速度上限が異なると打撃線が曲がるため方向を保つ協調スルーレート制限、(3) 打点時刻のコミットメント（ヒステリシス）である。結果、打撃方向誤差は平均 0.12°（90% 点 0.22°）、打点誤差 3.9 mm になった。シュート防御ドリル（2–4.5 m/s）の成功率は追従制御 A がストレート 50%・バンク 60%、物理予測制御 B が 100%・94%。

6. AI の活用方法

- **D: Hybrid AI（物理 + 学習）** — PPO の方策が「戦略（防御／ストレート／左壁バンク／右壁バンク）× 狙い（ゴール線上 5 点）× 打球速度（3 段階）」を選び、物理プランナが実行する。入力は状態 15 次元（パック・自分・相手の位置速度、相対位置、ポゼッション時計）と物理特徴 12 次元（防御線通過予測、最速打点、各シュートの空きレーン幅、バンク可否）。判断はイベント駆動で、1 回の判断が 1 本のシュートに対応する。
- **C: 純粋な強化学習** — PPO がマレットの目標位置（2 次元連続値）を 25 Hz で直接出力する。物理予測を使わない比較対象。
- **適応（B*, D*）** — 自由飛行区間の $\ln|v|$ 回帰で減衰率、側壁バウンドで反発係数をオンライン同定し、事前値と証拠重みで混合して予測モデルを更新する。センサノイズには物理モデル予測付きの α-β フィルタ（衝突でリセット）を用いる。

7. 学習手法

Stable-Baselines3 の PPO（MlpPolicy）。純粋 RL は 256×256、Hybrid は 128×128 の方策・価値ネットワーク。カリキュラムは Stage 1 静止パック打撃 → 2 直線シュート防御 → 3 壁反射シュート防御 → 4 単純な相手との対戦 → 5 複数相手 + ドメインランダム化 → 6 未知条件評価（学習に不使用）。ドメインランダム化は減衰率 0.05–0.35、壁反発 0.75–0.92、マレット反発 0.6–0.8、相手速度 0.8–1.1 倍、観測ノイズ（位置 3 mm・速度 0.03 m/s）。純粋 RL の報酬はゴール ±10、打撃 +0.3、枠内打撃 +0.7、防御 +1、自陣方向への打撃 −0.5、エネルギー・範囲外ペナルティと、最適方策を変えないポテンシャル型整形。Hybrid はゴール ±10、枠内 +0.5、防御 +0.5、自陣方向 −0.5 のみ。学習はすべてローカル CPU（Dev Container）で行い、Colab の計算ユニット消費は 0 である（理由は 14 章）。

学習	総ステップ	学習時間	速度	並列環境
ppo_direct_s0	10,004,480	50 分	3321 /s	10
ppo_hybrid_s0	1,000,960	43 分	387 /s	10

表2 学習の計算コスト（results/compute_cost.json）。Hybrid のステップ = 判断（ショット）回数。

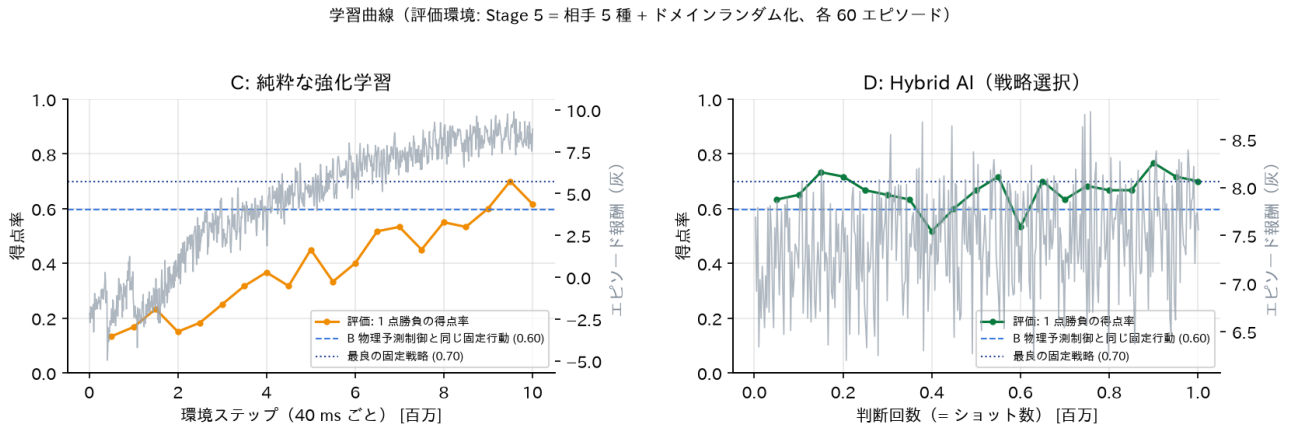


図2 学習曲線（Stage 5 環境での 1 点勝負の得点率と報酬）。破線は B と同じ固定行動、点線は最良の固定戦略（results/env_baselines.json）。

8. 実験設定

制御器 A 追従制御、B 物理予測制御、B+ 物理 + ルール（相手から最も遠いコースを選ぶ幾何ヒューリスティック）、B* 物理 + 適応、C 純粋 RL、D Hybrid AI、D* Hybrid AI + 適応 を、相手 5 種 × 条件 4 種（標準、学習分布のランダム化、未知: 高摩擦・低反発、未知: 低摩擦・高反発。未知条件は相手速度 1.15–1.3 倍・観測ノイズ 2 倍以上も含む）× 20 試合（各 60 秒）で評価した（計 2800 試合、評価用シード 10000 番台は学習と重複しない）。指標は勝率、得点・失点、防御成功率（自陣ゴールに入る軌道のシュートを止めた割合）、ラリー長、予測誤差、判断時間。

9. 実験結果

制御器	勝率	得点/60s	失点/60s	防御成功率	予測誤差 0.5s	判断時間
A: Reactive	33% (±9)	2.76	3.38	52.6%	—	0.01 ms
B: Physics	80% (±8)	7.26	0.21	98.4%	2.4 cm	0.09 ms
B+: Physics+Rule	91% (±6)	7.09	0.32	95.7%	2.7 cm	0.10 ms
B*: Physics+Adapt	80% (±8)	7.29	0.24	98.2%	2.6 cm	0.10 ms
C: PPO (pure RL)	97% (±3)	4.83	0.55	93.7%	—	0.21 ms
D: Hybrid AI	100% (±0)	9.20	0.38	95.3%	2.4 cm	0.15 ms
D*: Hybrid AI+Adapt	100% (±0)	9.83	0.33	95.4%	2.8 cm	0.17 ms

表3 標準条件の結果（相手 5 種 × 20 試合の平均。勝率の括弧は 95% 信頼区間の半幅）。

学習曲線（図2）では、純粋 RL の 1 点勝負の得点率は 0.13 から 0.62–0.70 まで上昇した。Hybrid は物理実行器のおかげで学習初期から 0.63 と高く、終盤は 0.70–0.77 と、相手プール全体の集計では最良の固定戦略（0.70）と同程度だった。差が明確に現れるのは相手別の試合評価である。標準条件では、物理予測制御 B が勝率 80%・平均 7.26 点/60 秒に対し、Hybrid AI（D）は勝率 100%・9.20 点だった（同一の相手・シードでの対応のある比較で、得失点差 +1.77、95% 信頼区間 [+0.98, +2.54]）。人手設計の戦略選択ルール B+（勝率 91%）に対しても全条件で得失点差 +1.26 [+0.95, +1.57] と上回った。最も差が大きいのは守備型の相手で、B は 0 点（全試合引き分け）だったのに対し、D は 5.65 点・勝率 100% だった。図7 のとおり、D は静止相手には右壁バンクショット（89%）、守備型にはストレート（53%）を多用するなど、相手ごとに戦略を変えている。純粋な強化学習 C は勝率 97% と高いが得点は 4.83 点と少なく、長いラリーで守り勝つスタイルを獲得した。学習に使っていない未知条件（高摩擦・低反発、低摩擦・高反発、相手 1.15–1.3 倍速、観測ノイズ 2 倍以上）では、D の勝率は 70% / 78% で B（71% / 79%）と同等にとどまったが、オンライン同定を加えた D* は 82% / 82% に向上し、0.5 秒先の予測誤差も 9.3→4.2 cm、11.8→6.9 cm と約半分になった（全条件で D*–D の得失点差 +0.61 [+0.34, +0.88]）。平均判断時間はすべて 0.25 ms 未満で、50 Hz の制御周期に対して十分小さい。

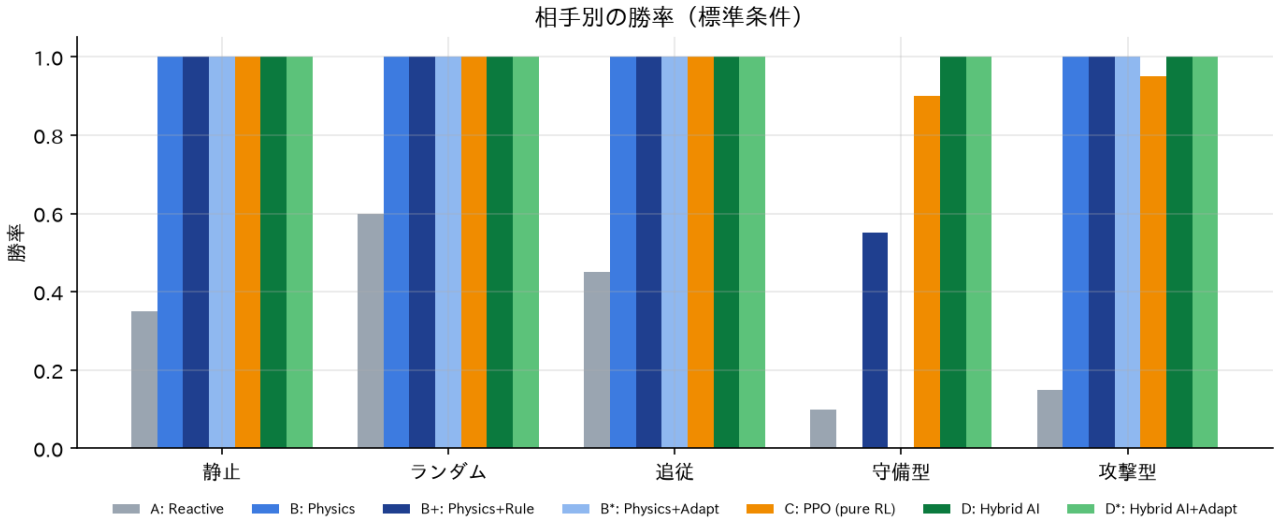


図3 相手別の勝率（標準条件）。

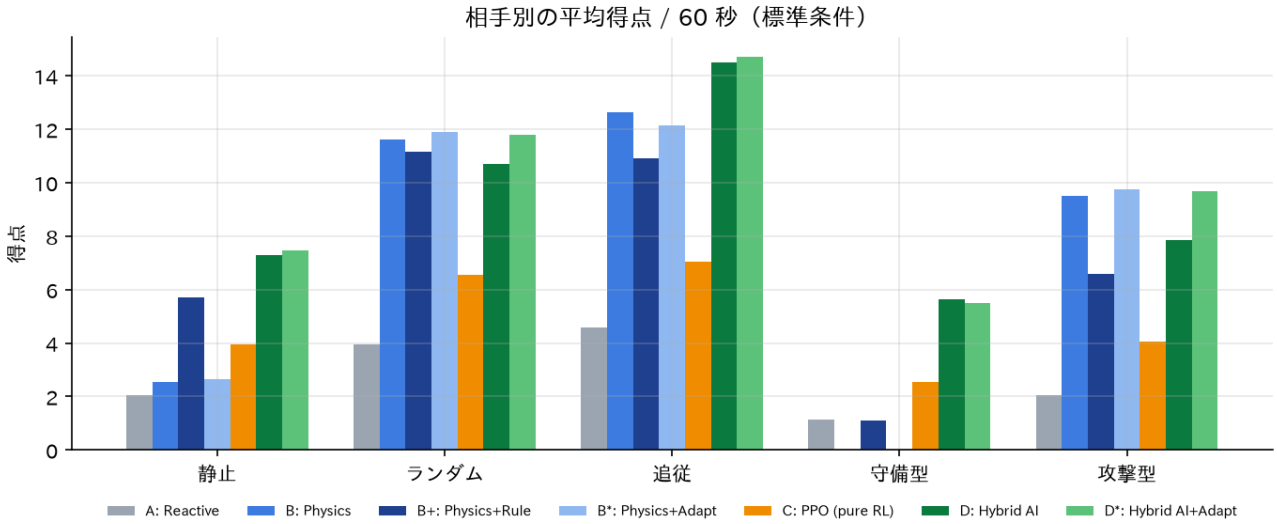


図4 相手別の平均得点（標準条件）。

制御器	標準	ランダム化	未知: 高摩擦	未知: 高反発
A: Reactive	0.40 / -0.62	0.42 / -0.63	0.59 / +0.49	0.34 / -1.95
B: Physics	0.90 / +7.05	0.94 / +5.40	0.84 / +2.23	0.82 / +3.35
B+: Physics+Rule	0.95 / +6.77	0.92 / +4.76	0.79 / +1.85	0.81 / +3.08
B*: Physics+Adapt	0.90 / +7.05	0.97 / +5.66	0.83 / +2.38	0.89 / +3.81
C: PPO (pure RL)	0.98 / +4.28	0.98 / +4.84	0.89 / +2.58	0.86 / +3.24
D: Hybrid AI	1.00 / +8.82	1.00 / +6.62	0.81 / +2.42	0.82 / +3.65
D*: Hybrid AI+Adapt	1.00 / +9.50	0.99 / +7.39	0.90 / +2.92	0.88 / +4.12

表4 条件別の 勝ち点率 / 平均得失点差（相手 5 種 × 20 試合）。

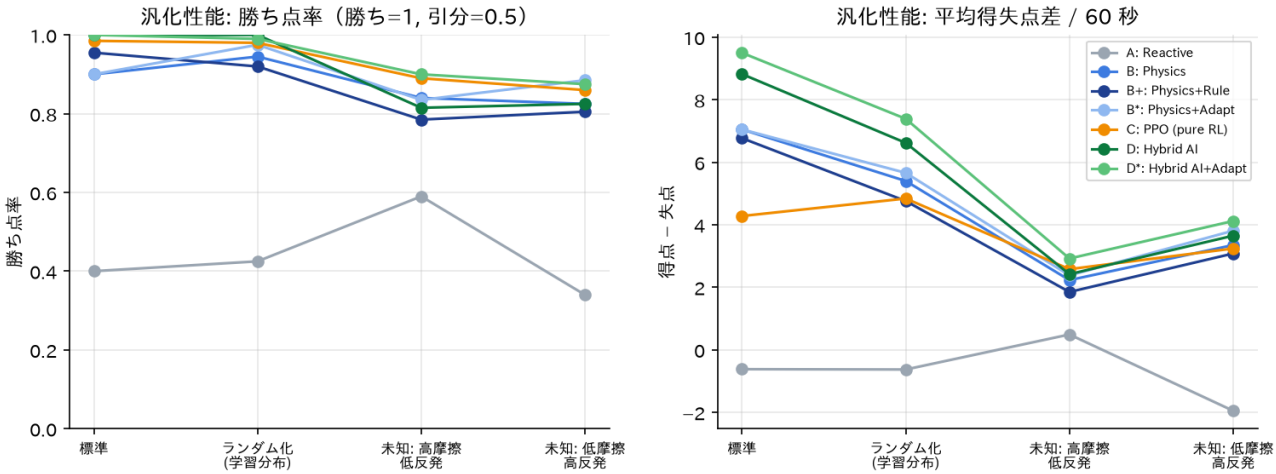


図5 未知条件への汎化（勝ち点率と得失点差）。

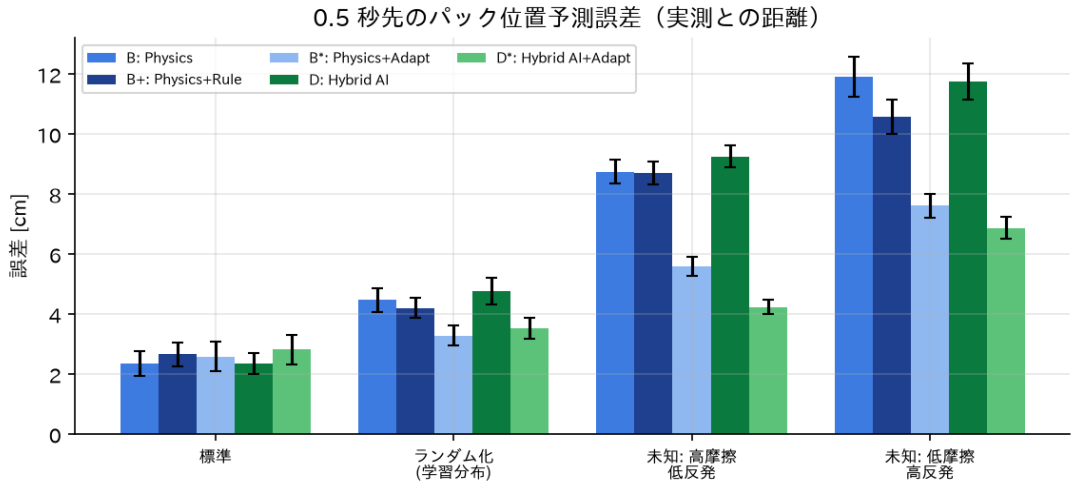


図6 0.5 秒先の予測誤差（物理予測を使う制御器）。適応付き（*）で未知条件の誤差が約半分。

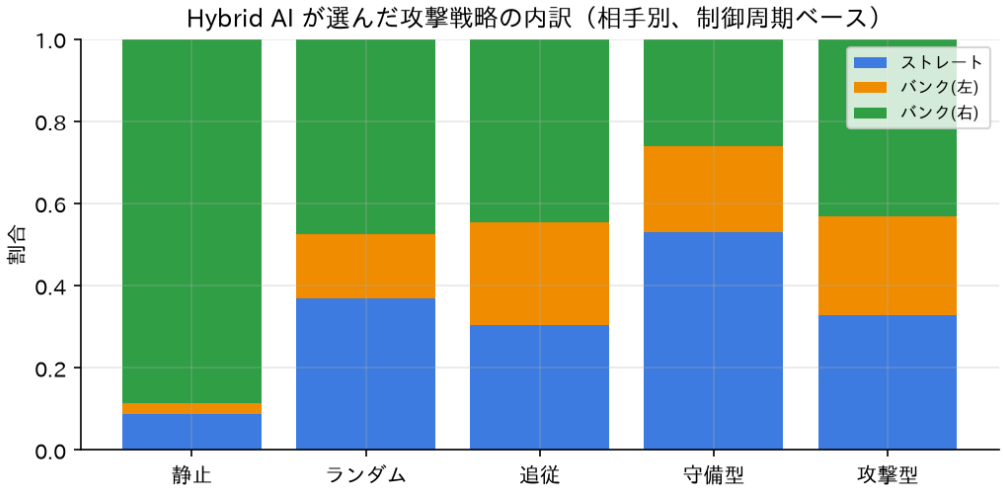


図7 Hybrid AI が相手ごとに選んだ攻撃戦略の割合。

10. 技術的工夫

- **反発係数の正確な再現**: MuJoCo の dampratio 方式で発散する問題を、接触ばねの固有振動数を固定する定式化で解決し、目標反発係数を ±0.01 で再現（表1）。
- **平面拘束 + 同一高さの球衝突形状**: 接触法線を厳密に水平にし、解析的な予測モデルとシミュレータの整合性を確保。
- **多重レート制御と協調スルーレート制限**: 判断 50 Hz・軌道生成 250 Hz・サーボ 1 kHz。軸ごとに加速度上限が違っても打撃線が曲がらない。
- **時刻付き打撃指令**: 低レベル側で指定時刻に加速を開始し、打撃方向誤差を 10–30° から平均 0.1° に改善。
- **ノイズに強い打点コミットメント**: 観測ノイズで計画が毎周期やり直される不具合を、許容幅付きコミットメントと静止判定で解消（ノイズ下の得点 2.75→10.75 点/60 秒）。

- **イベント駆動の意思決定**: 10 Hz で戦略を選ばせると探索ノイズが実行を壊し基準以下だったため、「実行器が空いた時だけ判断 = 1 判断 1 ショット」に変更して学習を成立させた。
- **物理特徴量**: 防御線通過予測・最速打点・各シュートの空きレーン幅・バンク可否を方策の入力に与え、物理の知識で学習を効率化。
- **オンライン同定 + α-β フィルタ**: 減衰率と反発係数を試合中に推定し、未知テーブルでの予測誤差を約半減。
- **人間らしい不完全な相手**: 反応遅延・速度・誤った物理モデルを持たせ、意味のある比較評価を可能にした。
- **再現性**: Dev Container（公式 devcontainer CLI で検証）、依存関係の完全固定、シード付き決定的リプレイ、CI、全図表・動画・本資料をスクリプトで再生成。

11. 考察

研究課題に対する答えは「条件付きで Yes」である。(1) 対戦性能: 物理予測制御の上で強化学習が戦略を選ぶ Hybrid AI は、同じ実行器を使う従来方式と人手設計ルールの両方を統計的に有意に上回った。特に守備型の相手には、物理予測だけでは 1 点も取れなかったが、学習によってストレートとバンクの使い分けや狙い・速度の組み合わせを見つけ、全勝した。これは「正確に打つ」物理と「どこへ打つか」の学習が補完的であることを示す。(2) 未知環境への適応: 学習（ドメインランダム化）だけでは学習分布外の物理条件で従来方式と同程度にとどまり、学習した戦略の価値は実行の精度に依存する。物理パラメータをオンライン同定して予測を補正すると、未知条件の勝率が 4–12 ポイント改善し、予測誤差が約半分になった。すなわち適応には「学習による戦略」と「物理モデルの同定」の両方が必要だった。(3) 純粋な強化学習は物理予測なしで勝率 97% に達したが、得点は従来方式より少なく、10M ステップを要した。Hybrid は 1M 判断（約 43 分）で上回る性能に到達しており、物理の構造を活かすことが学習効率と精度の両面で有利だった。一方で、Hybrid は攻撃的になる分だけ防御成功率がわずかに下がる（98.4%→95.3%）というトレードオフも見られた。

12. 制限事項

- すべてシミュレーション上の結果で、実機（カメラ遅延、摩擦の不均一、モーターの非線形性）での検証はしていない。
- 学習済み方策はそれぞれ 1 つの学習シードで、評価は各条件 20 試合（計 2,800 試合）。学習シード間のばらつきは未評価。
- 相手はスクリプト制御（反応遅延・誤差付き）で、自己対戦による強い相手との学習・評価は行っていない。
- 予測モデルはゴールポスト角との衝突を近似しておらず、ソフト接触のめり込みにより 0.5 秒先で約 2 cm の誤差が残る。
- Google Colab は CLI の検証と承認ゲート付きの手順整備まで（OAuth 認証が必要なため未実行）。本研究の負荷は CPU 律速で、ローカル実行の方が速かった。

13. 今後の展望

- 自己対戦（リーグ学習）で相手も学習させ、より強い相手への汎化を評価する。
- 予測モデルの残差（ゴールポスト・ソフト接触）をニューラルネットで補正する残差学習。
- カメラ画像からのパック検出と遅延を含む観測モデルで、実機への移行（Sim-to-Real）を目指す。
- 防御プリミティブも学習対象に含めた階層型強化学習と、複数シードでの統計的評価。

14. 再現性

すべて VS Code Dev Container（Python 3.11.17, MuJoCo 3.15.0, Gymnasium 1.4.0, Stable-Baselines3 2.9.0, PyTorch 2.14.1 CPU）で実行でき、依存関係は requirements/lock.txt に固定した。学習: `python scripts/train.py --config configs/training/ppo_hybrid.yaml`、評価: `python scripts/evaluate.py --config configs/evaluation/main.yaml`、図・動画・本資料: `make figures video report`。テスト（pytest 30 件以上）と Ruff を CI で実行する。Colab は公式 google-colab-cli 0.7.4 の手順（承認ゲート付き）を用意したが、MLP の PPO は環境計算が律速で CPU 並列の方が速いため本研究では使用していない。